

Assignment 2

Exercise groups 10

Magnus Fidje, Oskar Støre and Håkon Skramstad

Exercise 1

1.1

The algorithm given in the code snippet below, determines the h-index. One prerequisite for the algorithm is that the dictionary is sorted beforehand in descending order

```
In [1]: citations = {
    "Reproducibility vs. replicability: a brief history of a confused terminology":
    "The scientific case for brain simulations": 225,
    "Towards reproducible descriptions of neuronal network models": 222,
    "Noise in integrate-and-fire neurons: from stochastic input to escape rates": 1
    "Sustainable computational science: the ReScience initiative": 190,
    "Spiking network simulation code for petascale computers": 165,
    "Exact subthreshold integration with continuous spike times in discrete-time ne
    "Parallel and Distributed Programming-Efficient Parallel Simulation of Large-Sc
    "Markov analysis of stochastic resonance in a periodically driven integrate-and
    "NEST 2.10.0": 78,
    "Dynamics of self-sustained asynchronous-irregular activity in random networks
    "Learning from the past: approaches for reproducibility in computational neuros
    "Meeting the memory challenges of brain-scale network simulation": 69,
    "NEST 2.12.0": 64,
    "Connectivity concepts in neuronal network modeling": 55,
    "NEST: The Neural Simulation Tool.": 52,
    "Constructing Neuronal Network Models in Massively Parallel Environments": 48,
    "NEST 2.18.0": 40,
    "NineML: the network interchange for neuroscience modeling language": 39,
    "NEST 2.20.1": 36,
    "Stochastic resonance in neuron models: Endogenous stimulation revisited": 36,
    "Extended difference-of-Gaussians model incorporating cortical feedback for rel
    "Linear mechanistic models for the dorsal lateral geniculate nucleus of cat pro
    "NEST 2.14.0": 28,
    "NEST by Example: An Introduction to the Neural Simulation Tool NEST": 25,
    "Simplicity and efficiency of integrate-and-fire neuron models": 25,
    "Aspects of signal processing in noisy neurons": 25,
    "Visualizing neuronal network connectivity with connectivity pattern tables": 2
    "Firing-rate models capture essential response dynamics of LGN relay cells": 23
    "Response of the difference-of-Gaussians model to circular drifting-grating pat
    "Creating, documenting and sharing network models": 21,
    "NEST 2.16.0": 20,
    "Firing-rate models for neurons with a broad repertoire of spiking behaviors":
    "Escape rate models for noisy integrate-and-free neurons": 19,
```

```

    "NEST 3.0": 18,
    "NEST 3.3": 13,
    "Reproducibility and Rigour in Computational Neuroscience": 13,
    "Signal processing by means of noise": 13,
    "Genetic algorithm for optimization of models of the early stages in the visual
    "NEST 2.8.0": 11,
}

h_score = 0

for i, (key, value) in enumerate(citations.items()):
    if i + 1 <= value:
        h_score += 1
    else:
        print("Article number:",(i+1), "'", key, "'", "not counted. Cited", value, "
        break

h_score

```

Article number: 26 ' Simplicity and efficiency of integrate-and-fire neuron models ' not counted. Cited 25 times

Out[1]: 25

1.2

The worst-case for the algorithm is:

$$T(n) = \Theta(n)$$

This is because the algorithm would need to search through the entire dictionary.

For the average-case of the algorithm: Assuming the h-score is found in the middle of the dictionary the runtime would be $n/2$, giving the same complexity of worst-case:

$$T(n) = \Theta(n)$$

Exercise 2

```

LIST-INSERT(L, x, y)
1  y.prev = x
2  if x == NIL
3      y.next = L.head
4      if L.head ≠ NIL
5          L.head.prev = y
6      L.head = y
7  else
8      y.next = x.next
9      if x.next ≠ NIL
10         x.next.prev = y
11     x.next = y

```

Runtime

Runtime for this algorithm would be:

$$T(n) = \Theta(1)$$

This is because it get direct accses to where to insert the elment, it does not need to search at all

Excerise 3

```

Delete-Singly(L, k)
    if L.head == NIL
        error "list is empty"
    if L.head.key == k
        L.head = L.head.next
        return
    x = L.head
    while x.next != NIL
        if x.next.key == k
            x.next = x.next.next
            return
        x = x.next
    error "k not found"

```

Runtime

Runtime for this algorithm would be on average:

$$T(n) = \Theta(n)$$

Because it would be a linear search

Excerise 4

4.1

```

Hash-Delete-Efficient(T, k, j)
    element = h(k, j)
    T[element] = DELETED

```

This algorithm will find the elemented to be deleted, and set it's value to DELETED. This will requiere less data movment to fill the hole as the question asked. But this algorithm will result in that linear-searching will be very inefficient

4.2

Hash Tabel

$= 0$

0	A
1	C
2	B
3	E
4	
5	F
6	G
7	
8	H
9	
10	D

$(0, A) \Rightarrow j = h(0, 0)$
 $= (0 + 0) \bmod 11$
 $\Rightarrow 0$ available

$(2, B) \Rightarrow j = h(2, 0)$
 $= (2 + 0) \bmod 11$
 $\Rightarrow 2$ available

$(1, C) \Rightarrow j = h(1, 0)$
 $= (1 + 0) \bmod 11$
 $\Rightarrow 1$ available

$(0, D)$ ~~for $i=0$ $j=0$ this is no available spot~~
 $\Rightarrow j = h(0, 0)$
 $= (0 + 0) \bmod 11$
 $\Rightarrow 0$ not available
 $i += 1$
 $j = h(0, 1)$
 $= (0 + -1) \bmod 11$
 $= 10$ available

$(3, E)$ $3 \bmod 11 \Rightarrow 3$ available
 $(5, F)$ $5 \bmod 11 \Rightarrow 5$ available

$(2, G)$ $\Rightarrow j(2, 0) \Rightarrow 2$ not available
 $i += 1 \Rightarrow j = h(2, 1)$
 $= (2 + -1) \bmod 11$ not available
 $i += 1 \Rightarrow j = h(2, 2)$
 $= (2 + 0) \bmod 11$ available

$(6, H)$: $h(6, 0)$ not available
 $i += 1 \Rightarrow h(6, 1) = (6 + -1) \bmod 11$ not available
 $i += 1 \Rightarrow h(6, 2) = (6 + 0) \bmod 11$ not available
 $i += 1 \Rightarrow h(6, 3) = (6 + 1) \bmod 11$
 $\Rightarrow 8$ available

Probe Sequence

$h(k, j) \rightarrow h(k, j+1) \dots h(k, j+n)$

k -values

	0	1	2	3	4	5	6	
i -value	0	0	1	2	3	4	5	6
1	10	0	1	2	3	4	5	
2	4	5	6	7	8	9	10	
3	2	3	4	5	6	7	8	
4	5	6	7	8	9	10	0	
5	8	7	6	5	4	3	2	

$h(k, i)$

Exercise 5

```
In [2]: def find_overlapping(l):
overlapping_dict = {}

for i, (start, end) in enumerate(l):
    for start_other, end_other in l[i+1:]:
        if start_other < end:
```

```

        if (start, end) not in overlapping_dict:
            overlapping_dict[(start, end)] = []
            overlapping_dict[(start, end)].append((start_other, end_other))
        else:
            break

    return overlapping_dict

def print_overlapping(overlaps):
    for key, value in overlaps.items():
        print(key, "overlaps with:", value)

L = [(2,5), (6,9), (7,11), (10,15), (12,14), (13,17), (19,21)]
print_overlapping(find_overlapping(L))

```

```

(6, 9) overlaps with: [(7, 11)]
(7, 11) overlaps with: [(10, 15)]
(10, 15) overlaps with: [(12, 14), (13, 17)]
(12, 14) overlaps with: [(13, 17)]

```

This algorithm assumes the meetings are sorted by the start time.

The runtime of the algorithm would be in the worst case. The reason for that would be that it will check all meetings in the first for-loop. The inner for loop will go through (n-i) meeting. This will result in the runtime:

$$T(n) = \Theta(n^2)$$

In best case, no meeting are overlapping. The outer for loop will check n meeting, and the break in the inner for loop will check the i+1 meeting. Resulting in a runtime of:

$$T(n) = \Theta(n)$$